CS50x 2026 - Lecture 0 Notes - Scratch

By Wisdom

Unary, Binary, and Decimal

Computer science is the study of information; it's about problem-solving.

The problem is usually described in three forms: the **input**, the black middle box, and finally the **output**.

Computers use 0s and 1s. The forms are:

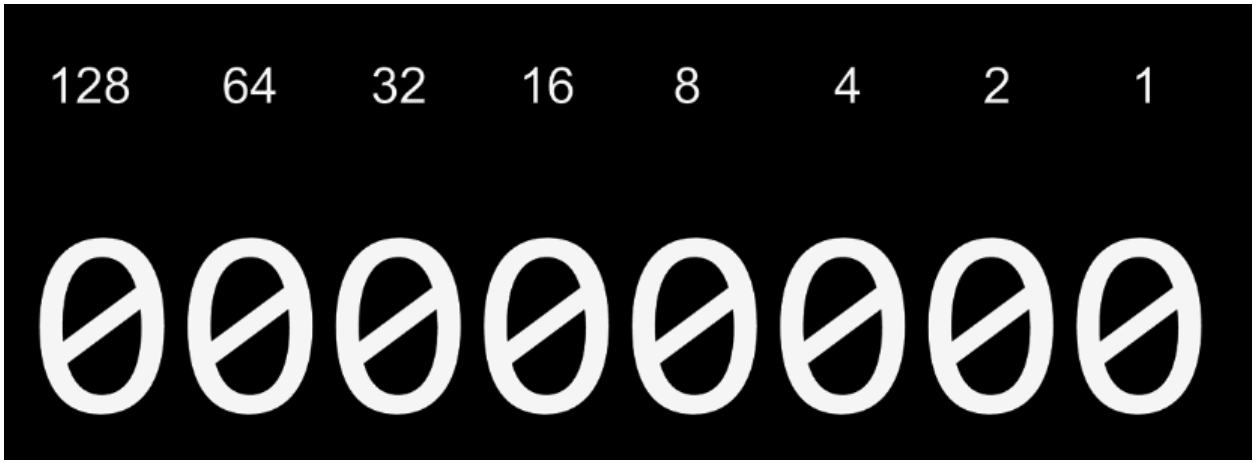
- **Unary Notation:** This is Base 1.
- **Binary Notation:** This is Base 2.

In a human hand, one hand can count up to 31 using binary notation. To understand better, think of each finger in the hand as holding a different value, like the thumb, middle finger, pinky, and so on. To better understand, follow this example:

Imagine each finger is like a tiny light switch that can be **off** or **on**. When a finger is down, it means **off (0)**, and when it is up, it means **on (1)**. Each finger is worth a different number: your thumb is worth 1, the next finger 2, then 4, then 8, then 16. If you lift fingers, you add their numbers together; for example, lifting the thumb (1) and the next finger (2) makes 3. By mixing which fingers are up, you can make every number from 0 to 31 using just one hand.

A **bit** is a single binary digit, either a 0 or a 1, or simply **ON (1)** or **OFF (0)**. Why? Because electricity is simple either flowing or not; the value itself is not considered. Computers have switches, not bulbs inside, and they are called **transistors**.

In the real world, we use Base 10, which goes from 0 to 9. Since using 1, 2, or 3 bits is not very practical, it's more common to use **bytes**. So, 1 byte is **8 bits**.



Think of a **bit** like a tiny switch that can be **off (0)** or **on (1)**. A **byte** is just **8 of those switches together**. Each switch has a value that doubles as you move along: **1, 2, 4, 8, 16, 32, 64, 128**. When all the switches are on, you add all those numbers together: 1 + 2 + 4 + 8 + 16 + 32 + 64 + 128 = **255**. So when all 8 bits are turned on (11111111 in binary), they equal **255** in normal numbers.

256 means 8 of something, and this is key because they are recurrent in computer science. 256 back in the day could only show 256 colors because they were using 8 bits.

Now in the world today, we use up to 32 & 64 bits, and you can get up to 4 billion, including both negative and positive numbers.

ASCII

This deals with alphabets in computer science, how to represent them in low-level

The secret is to give these new values an identity, like how to treat some patterns of 0s and 1s.

And to represent the letter A, its 65 -> 01000001, and 66 is B, 67 is C

Computers only understand numbers, not letters, so people needed a **shared agreement** on which numbers should represent which characters. In the 1960s, engineers created **ASCII (American Standard Code for Information Interchange)** so different computers and devices could send text and still understand each other. They assigned numbers to characters in a simple, organized way, putting control symbols first (like newline and tab), then numbers, then uppercase letters, then lowercase letters. Uppercase **A** starts at 65, and the rest follow in

order (B = 66, C = 67, etc.) to make sorting and processing text easier. It was not because letters naturally equal those numbers, but because people agreed on a standard so computers could communicate reliably.

It's just a mapping.

0	NUL	16	DLE	32	SP	48	0	64	@	80	P	96	`	112	p
1	SOH	17	DC1	33	!	49	1	65	A	81	Q	97	a	113	q
2	STX	18	DC2	34	"	50	2	66	B	82	R	98	b	114	r
3	ETX	19	DC3	35	#	51	3	67	C	83	S	99	c	115	s
4	EOT	20	DC4	36	$	52	4	68	D	84	T	100	d	116	t
5	ENQ	21	NAK	37	%	53	5	69	E	85	U	101	e	117	u
6	ACK	22	SYN	38	&	54	6	70	F	86	V	102	f	118	v
7	BEL	23	ETB	39	'	55	7	71	G	87	W	103	g	119	w
8	BS	24	CAN	40	(	56	8	72	H	88	X	104	h	120	x
9	HT	25	EM	41	)	57	9	73	I	89	Y	105	i	121	y
10	LF	26	SUB	42	*	58	:	74	J	90	Z	106	j	122	z
11	VT	27	ESC	43	+	59	;	75	K	91	[	107	k	123	{
12	FF	28	FS	44	,	60	<	76	L	92	\	108	l	124	|
13	CR	29	GS	45	-	61	=	77	M	93	]	109	m	125	}
14	SO	30	RS	46	.	62	>	78	N	94	^	110	n	126	~
15	SI	31	US	47	/	63	?	79	O	95	_	111	o	127	DEL

A cool trivia note is that lowercase letters are all 32 away from the uppercase letters.

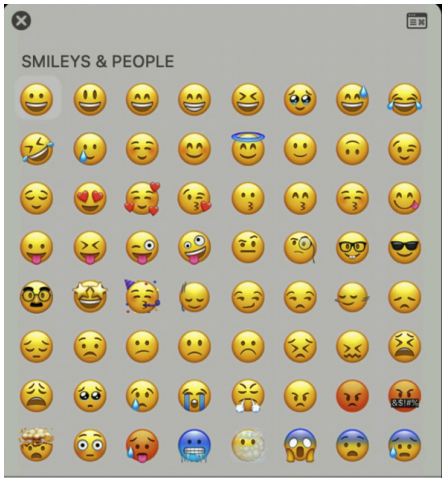
In ASCII, uppercase and lowercase letters are arranged so that lowercase letters are exactly **32 numbers higher** than their uppercase versions. The number **32** corresponds to turning on the **2^5 (the 32 column)** in binary.

For example, uppercase **A** is **65**, which in binary is **01000001**. Lowercase **a** is **97**, which is **65 + 32**, and in binary it becomes **01100001**. The only difference between them is that the **32-bit (the sixth bit from the right)** changes from **0 to 1**.

So, to change uppercase **A** to lowercase **a**, you just turn on the 32-bit, and to change lowercase back to uppercase, you turn that same bit off. This works for all English letters because ASCII was designed to keep uppercase and lowercase letters exactly 32 apart.

Unicode

Due to the large number of variants in letters, like the Chinese glyphs, Igbo, Yoruba, and many other languages, and the space running out, we use a superset of ASCII called Unicode. They have rooms for emojis, too.



Even emojis are characters in different fonts.

Unicode is focused on figuring out a way to digitally represent all human languages, both past and present, in characters by using 32 bits, 16 bits per character.

The laughing emoji is the 4 billion...then why are they different in terms of devices like Apple and Telegram, because there's a standard, and these companies choose how to represent them for their customers.

Color

RGB. How do computers represent colors, red, blue, and green? The answer is simply Integers.

Computers represent colors using **numbers**, just like they represent letters with ASCII. In the **RGB color system**, every color is made by mixing three basic colors: **Red, Green, and Blue**. Each of these colors is represented by a **number from 0 to 255**, where 0 means "none of that color" and 255 means "full intensity." For example, pure red is **(255, 0, 0)**, pure green is **(0, 255, 0)**, and pure blue is **(0, 0, 255)**. By combining different numbers for red, green, and blue, you can make **millions of colors**, because each channel is like a byte controlling how strong that color is.

In Photoshop and many other programs, colors are often shown in **hexadecimal (hex) format** instead of normal numbers. Hex is just another **way to write numbers**, like base 16 instead of base 10. Each color channel, Red, Green, and Blue, can go from **0 to 255** in decimal, which is **00 to FF in hex**. For example, pure red is **255, 0, 0** in decimal, and in hex it's **FF0000**. Hex just makes it shorter and easier to read, but under the hood, the computer is still using **integers from 0 to 255** to represent each color channel.

If you zoom deep into emojis and emojis, you see little dots like square shapes called pixels -> they are individual dots that collectively compose an image, and they are all distinct colors. This means we are using several bits to represent these individual colors.

Every **pixel** in an image is like a tiny colored dot. To make that pixel show a color, we use numbers for **Red, Green, and Blue**, the RGB system. If each color channel is **8 bits (1 byte)**, then one pixel needs **3 bytes** in total: one for red, one for green, and one for blue. For example, a pixel with full red, no green, and half blue would be **(255, 0, 128)**, which in binary is three separate 8-bit numbers. When you zoom out, millions of these colored pixels together form the full image, like an emoji or a photo. So basically, **each pixel is like a tiny packet of three numbers**, and the more pixels you have, the more detailed your image can be.

Representation

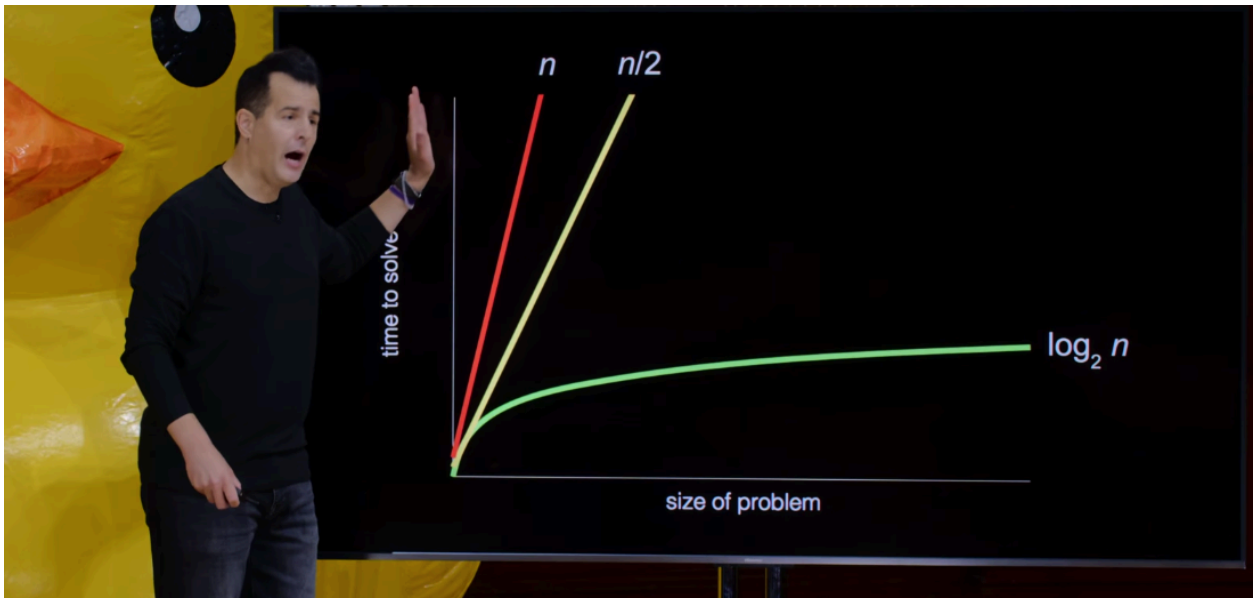
Everything on a computer starts with 0s and 1s, tiny switches that are either off or on. By putting these switches together, we can make numbers, and with numbers we can make letters, then colors, then pictures.

Movies are made by showing lots of pictures very fast, one after another. Our eyes and brain get tricked into thinking the pictures are moving, which is why movies were once called motion pictures.

Music is made by controlling three things at a time for each tiny piece of sound: how high or low the sound is (frequency), how loud it is (amplitude), and how long it lasts (duration). By combining these pieces, computers can create songs and sounds we can hear.

Algorithms

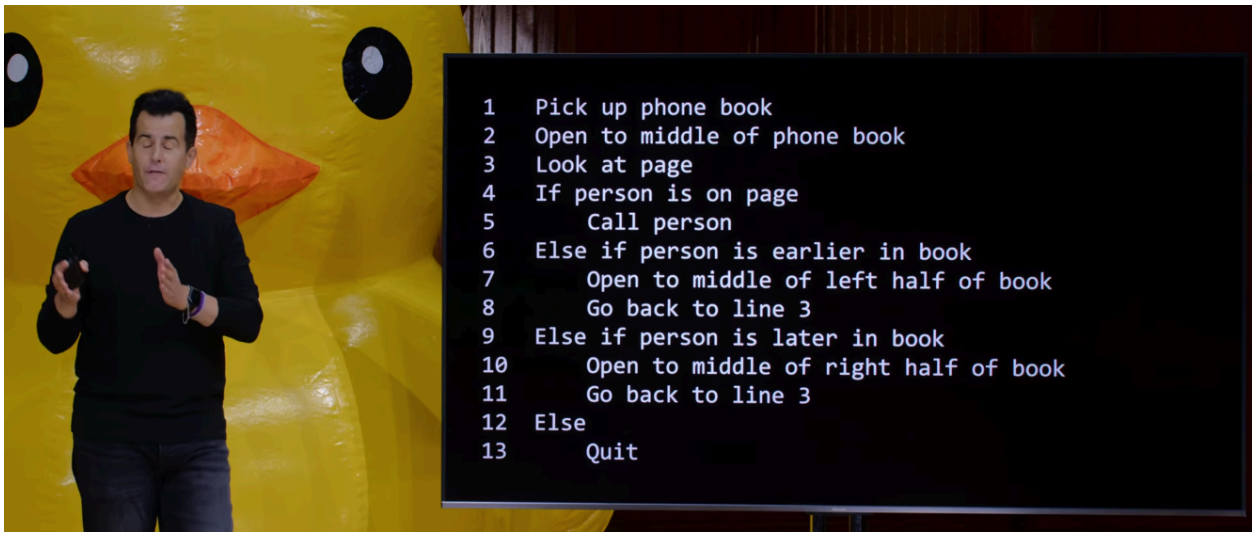
Algorithms are everywhere, like our phones, computers, and devices. Imagine you want to **find a number or a name in your contacts**. Instead of checking every single contact one by one, the algorithm can **look for patterns** as you type. For example, if you start typing **"Jo"**, the algorithm quickly scans the list and shows only contacts that **match those letters**, like "John" or "Jordan." As you type more letters or numbers, it **narrows down the results** faster, so you see exactly what you're looking for without waiting.



This graph above simply shows these:

1. **O(n) – Linear Search (1 of n)**
   If the algorithm just checks **every contact one by one**, it takes longer as the list grows. On a graph, this is a **straight diagonal line**: double the contacts, double the time. That's **linear time**.
2. **O(n/2) – Half the List on Average**
   If the algorithm is clever and can stop halfway on average (like if the number you're looking for is somewhere in the middle), it still grows roughly like O(n). On a graph, it looks **almost the same as O(n)**, just a bit lower because, on average, you check fewer items.
3. **O($\log_2$ n) – Binary Search**
   If the contacts are **sorted**, you can use a **binary search**: check the middle contact, then cut the search space in half each time. Each step reduces the remaining contacts by half, so even huge lists take only a few steps. On a graph, this curve **rises very slowly**, almost flat compared to O(n), because doubling the contacts only adds **one extra step**.

Pseudocode



Code is just a way to write algorithms so that a computer can understand and follow them. It's all about taking inputs, doing some steps, and giving outputs.

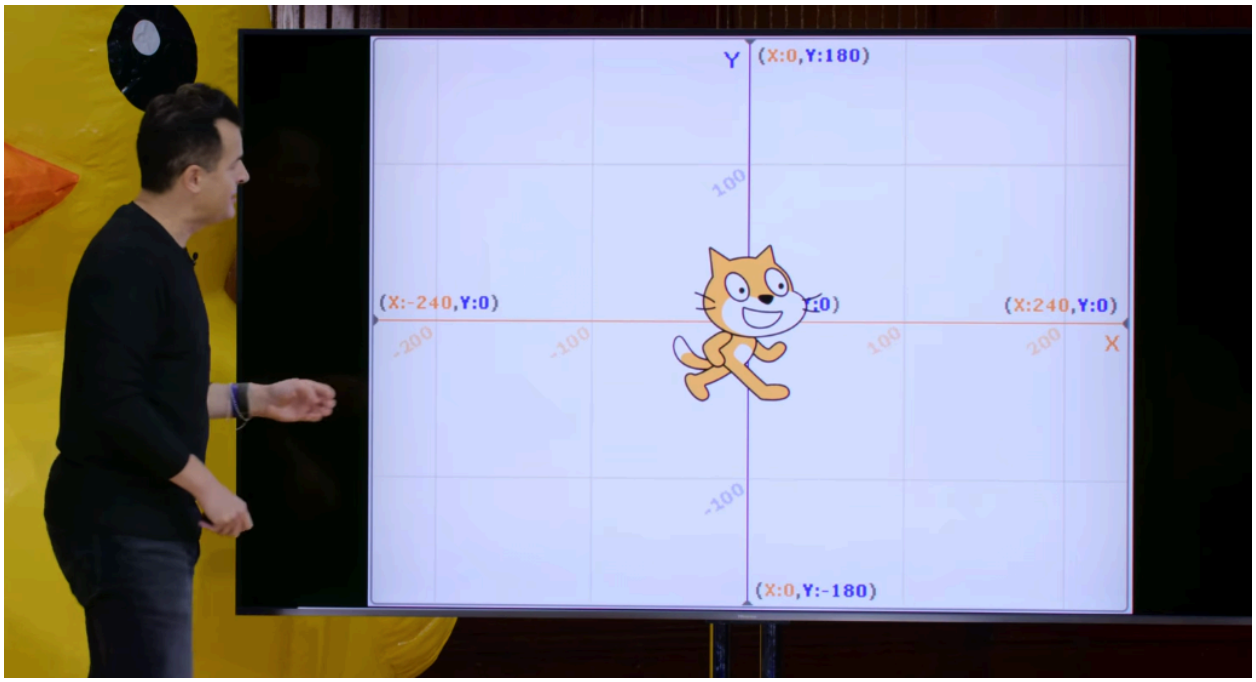
Pseudocode is like writing your algorithm in plain English (or any language you like) before turning it into real code. It's a step-by-step description of what the program should do, kind of like your thought process written down.

1. Functions are verbs that gets works done/action in your code
2. Conditionals are for giving a guided autonomy, do I go left, right, or do something else
3. Boolean expressions are questions with true, false, 1, or 0, or simply a binary state
4. Loop induces cyclical behaviours.

Companies like Intel, Nvidia, and AMD standardize what patterns of 0s and 1s represent what functionality, really low level stuffs.

The principles of abstraction are really the best thing to happen in computer science, standing on the shoulders of others. A compiler is a program that translates one language into another.

Scratch



Scratch is drag and drop form of programming languages.

Functions take arguments/parameters, which is an input to the function. A side effect is something that happens visually, which happens as a result of using a function.

A **function** is a piece of functionality implemented in code that carries out an **algorithm**. The algorithm is the concept, and the function is its **incarnation**. The **output** of a function is its side effect.

Functions can also take **arguments** (inputs) and **return a value**, which is something the computer sees but the user usually doesn't. The return value is stored in a **variable** so the program can use it later.

As for this lecture, I created a simple Game in Scratch.

Meow Game Instructions

Goal: Create a simple game where a cat **meows whenever the mouse hovers over it**.

Technical Overview:

1. The game uses a **forever loop** to constantly check for interactions. Without this loop, the program would only check once, and the meow wouldn't respond continuously.

2. The program detects **mouse coordinates** and compares them with the cat's position on the screen.
3. When the mouse is **hovering over the cat**, the program triggers a **meow sound**.
4. The loop ensures that this detection happens **frame by frame**, allowing real-time interactivity.

Implementation Steps:

1. Display the cat on the screen at a fixed position.
2. Start a **forever loop** that continuously:
    - Checks the current mouse position.
    - Compares it with the cat's boundaries.
    - Plays the meow sound if the mouse is over the cat.
3. End the loop only when the program is closed.

Key Concept: The **forever loop** is crucial because it makes the program **react continuously** to user input, mimicking real-time behavior. Without it, the program would not detect the hover after the first check.

Link: https://scratch.mit.edu/projects/1275980400

Note: I kept the game simple because I'm not particularly interested in using Scratch, and I already have a solid background in programming.